

Odoo (ERP/Warenwirtschaft) ↔ Skipily verbinden (Schritt für Schritt)

Voraussetzung: Lies zuerst `00-Skipily-Anbindung-Grundlagen.md` und halte deinen **Skipily-API-Schlüssel** bereit.

Warum Odoo? Odoo ist eines der weltweit meistgenutzten ERP-/Warenwirtschaftssysteme für kleine und mittlere Betriebe – es vereint Lager, Verkauf, Rechnungen und Versand in einem Werkzeug und lässt sich sehr gut anbinden. **Wichtig:** Die hier gezeigten Schritte funktionieren nach demselben Muster auch mit anderen ERP-Systemen wie **JTL-Wawi, Xentral, Microsoft Dynamics 365 Business Central oder SAP Business One** – überall gilt: *Produkte raus → Bestellungen rein → Versand zurück*.

Wir verbinden alles über das kostenlose No-Code-Werkzeug **Make.com**. So brauchst du weder einen ERP-Programmierer noch Serverzugriff.

Was am Ende funktioniert:

- Neuer/geänderter Artikel in Odoo → erscheint automatisch bei Skipily.
- Bestellung bei Skipily → wird automatisch als Verkaufsauftrag in Odoo angelegt.
- Lieferung/Versand in Odoo bestätigt → Status + Sendungsnummer gehen zurück an Skipily.

Plane **ca. 60 Minuten** ein (ERP hat ein paar Felder mehr).

 **Abkürzung – fertige Vorlage:** Ein fertiges Make-Template „Odoo ↔ Skipily“ gibt es in der öffentlichen Galerie (noch) nicht, aber wir liefern **importierbare Blueprints** mit (Ordner `blueprints/`), die den kompletten Skipily-Teil (URL, Header, JSON) vorbefüllen: `skipily-produkt-anlegen.json` (Teil A), `skipily-bestellung-empfangen.json` (Teil B), `skipily-versand-melden.json` (Teil C). **Import:** Neues Szenario → unten „...“ (**More**) → „**Import Blueprint**“ → Datei wählen. Danach nur noch `x-api-key` einsetzen, den Odoo-Baustein ergänzen und Felder zuordnen – genau das zeigen die Teile A–C unten. Die Blueprints funktionieren unverändert auch mit JTL, Xentral, Dynamics oder SAP B1.

Teil 0 – Vorbereitung (15 Min)

1. **Make.com-Konto** anlegen: <https://www.make.com> → „Get started free“.
2. **Skipily-API-Schlüssel** bereitlegen (siehe Grundlagen).
3. **Odoo-Zugang für Make** vorbereiten. Make verbindet sich mit deiner Odoo-Instanz über:
 - **Server-Adresse** deiner Odoo (z. B. `https://deinbetrieb.odoo.com`)

- **Datenbank-Name** (steht bei Odoo-Online meist im Login-Bereich; bei Bedarf Odoo-Support fragen)
- **Benutzername** (deine Odoo-Login-E-Mail)
- **API-Schlüssel** von Odoo: in Odoo oben rechts auf deinen Namen → „**Mein Profil**“ → **Reiter „Konto-Sicherheit“** → „**Neuer API-Schlüssel**“ → Name `Skipily` → **kopieren** und sicher speichern.

Nutzt du **kein** Odoo? Dann brauchst du vom jeweiligen ERP-Anbieter dasselbe Trio: **Server-Adresse, Benutzer, API-Schlüssel/Token**. Der Rest der Anleitung bleibt gleich – nur die Baustein-Namen in Make heißen dann z. B. „JTL“ oder „Dynamics“.

Jetzt hast du: Skipily-API-Schlüssel + Odoo-Zugang (Server, DB, Benutzer, API-Key).

Teil A – Artikel von Odoo zu Skipily schicken (20 Min)

Ziel: Neuer/geänderter Artikel in Odoo → automatisch bei Skipily.

1. In **Make.com** → „**Create a new scenario**“.
2. „+“ → „**Odoo ERP**“ suchen → Baustein „**Watch Records**“ (Datensätze beobachten).
 - „**Add**“ bei Connection → Server-Adresse, Datenbank, Benutzer, API-Schlüssel aus Teil 0 eintragen → **Save**.
 - **Model / Objekt:** `product.template` (das ist der „Artikelstamm“ in Odoo).
 - **Trigger:** „Created or updated“.
3. Rechts daneben „+“ → „**HTTP**“ → „**Make a request**“ (Skipily-Aufruf):
 - **URL:** `https://vcjwlyqkfkszumdrfvtm.supabase.co/functions/v1/products-api`
 - **Method:** `POST`
 - **Headers:**
 - `Authorization` → `Bearer eyJhbGci...` (öffentlicher Wert aus den Grundlagen)
 - `x-api-key` → **dein Skipily-API-Schlüssel**
 - `Content-Type` → `application/json`
 - **Body type:** `Raw` → **JSON (application/json)**
 - **Request content** (Odoo-Felder per Klick einsetzen):

```
{
  "name": "{{name aus Odoo}}",
  "sku": "{{default_code (interne Referenz) aus Odoo}}",
  "manufacturer": "{{Hersteller/Marke, falls gepflegt}}",
  "price": "{{list_price aus Odoo}}",
  "currency": "EUR",
  "stock_quantity": "{{qty_available aus Odoo}}",
  "in_stock": true,
  "weight_kg": "{{weight aus Odoo}}",
  "ean": "{{barcode aus Odoo}}",
  "description": "{{description_sale aus Odoo}}"
}
```

4. **Speichern** → „**Run once**“ → in Odoo einen Test-Artikel anlegen/ändern → Szenario läuft grün → Artikel erscheint bei Skipily.
5. Links unten „**Scheduling**“ auf **ON** (z. B. alle 15 Min).

Sauberes Aktualisieren (empfohlen für ERP): Lege in Odoo pro Artikel ein Zusatzfeld „Skipily-ID“ an und schreibe die von Skipily zurückgegebene `id` dort hinein. Beim nächsten Lauf: Ist die Skipily-ID vorhanden → **PUT** `.../products-api?id=DIE-ID` (Update); sonst **POST** (neu). In Make regelst du das mit einem **Router** + Filter „Skipily-ID ist leer?“. Für den Start genügt reines Anlegen.

Teil B – Skipily-Bestellungen als Verkaufsauftrag in Odoo anlegen (15 Min)

Ziel: Kauf über Skipily → automatisch ein **Verkaufsauftrag (sale.order)** in Odoo. Skipily „klingelt“ per **Webhook**.

1. In Make **neues Szenario**.
2. Erster Baustein „**Webhooks**“ → „**Custom webhook**“ → „**Add**“ → Name `Skipily-Bestellungen` → **URL kopieren**.
3. URL bei Skipily hinterlegen: **Provider-Portal** → **Einstellungen** → **Integrationen** → „**Webhook-URL für Bestellungen**“ → einfügen → speichern. (Kein Feld? URL an support@skipily.app.)
4. Zweiter Baustein „**Odoo ERP**“ → „**Create a Record**“:
 - **Model:** `sale.order` (Verkaufsauftrag). Kunde, Lieferadresse, Positionen aus dem Skipily-Webhook zuordnen. Artikel werden über die **SKU** (`default_code`) den Odoo-Artikeln zugeordnet – ggf. davor einen „Search Records“-Baustein einbauen, der zur SKU die Odoo-Artikel-ID findet.
5. **Speichern + Scheduling ON**.

6. **Test:** Test-Bestellung bei Skipily → Verkaufsauftrag erscheint in Odoo.

Skipily-Bestell-ID merken (`order_id` / `order_number` aus dem Webhook) → in ein Odoo-Feld am Auftrag schreiben (z. B. „Kundenreferenz“/ `client_order_ref`). Brauchst du in Teil C.

Teil C – Lieferung & Tracking an Skipily zurückmelden (15 Min)

Ziel: Bestätigst du in Odoo die Lieferung (Lieferschein „erledigt“, Sendungsnummer erfasst), sieht der Kunde bei Skipily „versandt“ + Sendungsnummer.

1. In Make **neues Szenario**.
2. Erster Baustein „**Odoo ERP**“ → „**Watch Records**“:
 - **Model:** `stock.picking` (Lieferungen/Lieferscheine).
 - Filter: nur mit **Status „done“ (erledigt)** und vorhandener **Sendungsnummer** (`carrier_tracking_ref`).
3. Zweiter Baustein „**HTTP**“ → „**Make a request**“:
 - **URL:** `https://vcjwlyqkfkszumdrfvtn.supabase.co/functions/v1/orders-api?id={{Skipily-Bestell-ID}}` (die ID, die du in Teil B am Auftrag gespeichert hast)
 - **Method:** `PUT`
 - **Headers:** dieselben drei wie in Teil A.
 - **Body** → **Raw** → **JSON:**

```
{
  "status": "shipped",
  "tracking_number": "{{carrier_tracking_ref aus Odoo}}",
  "tracking_url": "{{Tracking-Link, falls vorhanden}}"
}
```

4. **Speichern, Run once** (eine Lieferung auf „erledigt“ setzen), dann **Scheduling ON**.

Fertig! Kurz-Kontrolle

- ☐ Test-Artikel in Odoo → erscheint bei Skipily (Teil A)
- ☐ Test-Bestellung bei Skipily → Verkaufsauftrag in Odoo (Teil B)
- ☐ Lieferung in Odoo „erledigt“ → Skipily zeigt „shipped“ + Tracking (Teil C)

Wenn etwas klemmt

- **Make findet Odoo-Verbindung nicht:** Datenbank-Name exakt prüfen (Groß-/Kleinschreibung); API-Schlüssel neu erzeugen und einsetzen.
- **„401/403“ bei Skipily:** `x-api-key` oder `Authorization` -Wert falsch → exakt neu kopieren.
- **„400 – name required“:** Feld `name` im Body leer → Odoo- `name` neu zuordnen.
- **Bestellpositionen leer:** Die **SKU** in Odoo (`default_code`) muss zur SKU aus der Skipily-Bestellung passen → „Search Records“ nach SKU vorschalten.
- **Anderes ERP (JTL/Xentral/Dynamics/SAP):** Ersetze in Make die „Odoo“-Bausteine durch die deines Systems; die Skipily-Seite (URLs, Header, JSON) bleibt **exakt gleich**.
- **Hilfe:** support@skipily.app (gern mit Screenshot der Make-Fehlermeldung).